

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»
Кафедра вычислительной техники

Расчетно-графическая работа
По дисциплине «Технологии и методы программирования»
на тему: «Разработка простой файловой системы»

Вариант 7

Факультет: АВТФ
Группа: АВТ-441
Студент: Тютрин В.А

Преподаватель: Булатов А.Д

Новосибирск
2026

ОГЛАВЛЕНИЕ

ПОСТАНОВКА ЗАДАЧИ	1
ХОД РАБОТЫ.....	4
ВЫВОД.....	11

ВВЕДЕНИЕ

Актуальность темы. В условиях стремительной цифровизации образовательной среды вопросы эффективного управления учебным процессом приобретают особую значимость. Традиционные методы учета академической успеваемости, основанные на бумажных носителях или разрозненных электронных таблицах, постепенно исчерпывают свой потенциал. Подобные подходы характеризуются высокой трудоемкостью обработки данных, низким уровнем оперативности получения информации и существенным риском возникновения ошибок из-за человеческого фактора. Для современных высших и средних специальных учебных заведений критически важным становится внедрение специализированных программных комплексов, способных автоматизировать рутинные операции по сбору, хранению и анализу данных об успеваемости обучающихся. Разработка такого программного обеспечения позволяет не только оптимизировать рабочее время преподавательского состава и сотрудников учебных отделов, но и обеспечивает прозрачность образовательного процесса, формируя единую достоверную базу знаний о прогрессе каждого студента. Таким образом, создание автономной, надежной и удобной в использовании системы контроля успеваемости является актуальной задачей, решение которой способствует повышению качества административного менеджмента в сфере образования.

Целью данной расчетно-графической работы является проектирование и программная реализация полнофункционального приложения для автоматизации учета и анализа академической успеваемости студентов. Разрабатываемая система должна обеспечить централизованное хранение всей необходимой информации, предоставить интуитивно понятный интерфейс для взаимодействия с данными и реализовать механизмы целостности и защиты информации. Ключевым аспектом разработки является соблюдение принципов модульности и масштабируемости архитектуры, что позволит в дальнейшем расширять функциональные возможности приложения без необходимости существенной переработки базового кода.

Для достижения поставленной цели в ходе работы решается ряд взаимосвязанных **задач**. Первостепенное значение имеет организация надежного слоя доступа к данным с использованием реляционной базы данных. Выбор технологии хранения информации обусловлен необходимостью обеспечения структурной целостности данных, поддержки сложных связей между сущностями «Студент», «Учебная дисциплина» и «Оценка», а также гарантией сохранности информации при сбоях в работе приложения. В качестве средства взаимодействия с базой данных применяется технология JDBC (Java Database Connectivity), которая является стандартом де-факто для Java-приложений, работающих с реляционными СУБД. Особое внимание уделяется реализации паттерна DAO (Data Access Object), который позволяет абстрагироваться от конкретной реализации базы данных и изолировать SQL-запросы от бизнес-логики приложения.

Второй важной задачей является разработка пользовательского интерфейса, отвечающего современным требованиям эргономики и удобства использования. Приложение реализуется на базе фреймворка JavaFX, который предоставляет богатые возможности для создания графических интерфейсов с поддержкой стилизации через CSS, анимаций и реактивного программирования. Интерфейс строится по принципу разделения ответственности, где визуальное представление отделяется от логики обработки событий с помощью шаблона MVC (Model-View-Controller). Это обеспечивает чистоту кода и упрощает его тестирование и сопровождение. Реализация включает в себя создание многооконной структуры с системой авторизации, главным окном с навигацией по вкладкам и специализированными панелями для управления каждым типом сущностей. Третья задача заключается в обеспечении корректной работы бизнес-логики приложения. Система должна предоставлять полный цикл операций CRUD (Create, Read, Update, Delete) для всех управляемых сущностей. Это подразумевает возможность добавления новых записей о студентах и предметах, редактирования существующих данных, удаления устаревшей информации, а также фиксацию оценок с привязкой к конкретным датам. Важным аспектом является реализация механизмов валидации вводимых пользователем данных, предотвращающих внесение некорректной информации в базу данных, например, оценок, выходящих за допустимый диапазон, или дат в неверном формате. Кроме того, приложение должно обрабатывать исключительные ситуации, такие как ошибки подключения к базе данных или нарушения ссылочной целостности, информируя пользователя о проблемах в понятной форме.

Объектом исследования в данной работе выступают процессы сбора, хранения и обработки информации об академической успеваемости обучающихся в рамках учебного заведения. **Предметом исследования** являются методы и средства разработки десктопных приложений на языке программирования Java, а также технологии взаимодействия с реляционными базами данных посредством JDBC API.

В качестве **инструментальных средств** разработки выбран язык программирования Java версии 17 (или 21/23 в зависимости от настроек проекта), который сочетает в себе строгую типизацию, объектно-ориентированный подход и кроссплатформенность. Для управления зависимостями и сборки проекта используется Apache Maven, что позволяет автоматизировать процессы компиляции и подключения внешних библиотек. В качестве системы управления базами данных выбрана SQLite. Данный выбор обусловлен рядом преимуществ для локальных десктопных приложений: отсутствием необходимости развертывания отдельного серверного ПО, высокой скоростью работы благодаря хранению данных в одном файле, простотой резервного копирования и полной совместимостью со стандартом SQL. Использование SQLite позволяет создать полностью

автономное решение, которое может быть запущено на любом компьютере без предварительной настройки окружения.

Практическая значимость выполненной работы заключается в создании готового к использованию программного продукта, который может быть внедрен в реальный учебный процесс для замены устаревших методов учета. Разработанное приложение демонстрирует применение современных архитектурных подходов, таких как многослойная архитектура и паттерн DAO, которые являются фундаментальными для построения корпоративных информационных систем. Полученные навыки проектирования баз данных, разработки графических интерфейсов на JavaFX и организации взаимодействия между слоями приложения имеют высокую ценность для дальнейшего профессионального развития в области программной инженерии.

Структура отчета. Работа состоит из введения, основной части, включающей описание постановки задачи, хода выполнения работы и результатов тестирования, заключения и списка использованных источников. В основной части подробно рассматриваются этапы проектирования архитектуры приложения, реализации слоев доступа к данным и пользовательского интерфейса, а также приводятся листинги ключевых фрагментов кода, иллюстрирующие принятые технические решения.

ПОСТАНОВКА ЗАДАЧИ

1. Назначение разрабатываемого программного обеспечения

Программное обеспечение «Контроль успеваемости студентов» предназначено для автоматизации процессов учёта и анализа академической успеваемости обучающихся в высших и средних специальных учебных заведениях. Система призвана заменить устаревшие ручные методы ведения ведомостей и журналов, обеспечить оперативный доступ к информации об успеваемости, а также предоставить инструменты для анализа и формирования отчётности.

Внедрение данной системы позволит преподавателям и сотрудникам учебных отделов сократить временные затраты на заполнение документации, минимизировать вероятность ошибок при учёте оценок, а также оперативно получать актуальную информацию о текущей успеваемости как по отдельным студентам, так и по академическим группам в целом.

2. Цель и задачи разработки

Основной целью данной работы является создание функционального, надёжного и удобного в использовании приложения для комплексного контроля успеваемости студентов.

Для достижения поставленной цели необходимо решить следующие задачи:

- организовать централизованное хранение всех данных об учебном процессе с использованием реляционной базы данных, что обеспечит целостность, непротиворечивость и долговременную сохранность информации;
- реализовать удобный пользовательский интерфейс в виде консольного меню, позволяющий выполнять все основные операции без необходимости прямого взаимодействия с базой данных;
- обеспечить возможность добавления, редактирования и удаления информации о студентах с учётом всех необходимых атрибутов,

включая личные данные, принадлежность к учебной группе и форму обучения;

- предоставить механизмы для ведения перечня учебных дисциплин с указанием таких характеристик, как семестр изучения, трудоёмкость в часах и форма промежуточной аттестации;
- организовать процесс фиксации успеваемости, позволяющий преподавателям выставять и корректировать оценки студентов по каждой дисциплине, а также отслеживать дату выставления каждой оценки;
- реализовать средства аналитической обработки накопленных данных, включая автоматический расчёт среднего балла успеваемости для каждого студента и дисциплины;
- разработать функционал формирования сводных отчётов, которые могут быть использованы для дальнейшего анализа, печати или экспорта.

3. Требования к пользовательскому интерфейсу

Приложение должно предоставлять текстовый консольный интерфейс с иерархической системой меню. Пользователь взаимодействует с системой путём ввода цифровых команд, соответствующих различным операциям. Все информационные сообщения, предупреждения об ошибках и запросы на ввод данных должны быть представлены на русском языке. Интерфейс должен быть интуитивно понятным, с чёткими подсказками о допустимых действиях на каждом этапе работы.

4. Требования к программной реализации

Разработка приложения должна выполняться на языке программирования Java версии 23. В качестве системы управления базами данных должна использоваться PostgreSQL. Взаимодействие между приложением и базой данных необходимо организовать посредством технологии JDBC.

Программный код должен быть структурирован в соответствии с принципами многослойной архитектуры, что предполагает чёткое разделение на слой доступа к данным, слой бизнес-логики и слой представления. Такой подход обеспечит модульность, удобство сопровождения и возможность последующего расширения функциональности.

Также в рамках работы необходимо обеспечить обработку исключительных ситуаций, включая ошибки подключения к базе данных, нарушения целостности данных и некорректный ввод информации пользователем.

5. Ожидаемые результаты

По итогам выполнения работы должно быть создано работоспособное приложение, позволяющее полностью вести учёт успеваемости студентов в рамках одного учебного заведения. Система должна демонстрировать корректную работу при выполнении всех заявленных операций, обеспечивать сохранность данных между сеансами работы и предоставлять возможность получения наглядных отчётов об успеваемости.

ХОД РАБОТЫ

1) Подключение SQLite в `pom.xml`

Подключён драйвер SQLite и задано компилирование под JDK 17:

```
1 <properties>
2   <maven.compiler.release>17</maven.compiler.release>
3   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
4 </properties>
5 ...
6 <dependencies>
7   <!-- SQLite JDBC Driver -->
8   <dependency>
9     <groupId>org.xerial</groupId>
10    <artifactId>sqlite-jdbc</artifactId>
11    <version>3.46.1.3</version>
12  </dependency>
13  ...
14 </dependencies>
```

Рисунок 1 – Pom.xml

2) Подключение и инициализация базы данных (`DBConnection`)

Создание подключения к SQLite выполняется по URL с фиксированным форматом дат (`date_string_format`). Перед запуском UI вызывается `initializeDatabase()`, которая читает `schema.sql` и выполняет команды создания таблиц:

```
1 public class DBConnection {
2   // date_string_format гарантирует единый формат дат для SQLite JDBC
3   private static final String URL = "jdbc:sqlite:students.db?date_string_format=yyyy-MM-dd";
4
5   public static Connection getConnection() throws SQLException {
6     Connection connection = DriverManager.getConnection(URL);
7     try (Statement statement = connection.createStatement()) {
8       statement.execute("PRAGMA foreign_keys = ON");
9     }
10    return connection;
11  }
12
13  public static void initializeDatabase() throws SQLException, IOException {
14    try (Connection connection = getConnection();
15         Statement statement = connection.createStatement()) {
16      String schema = readSchema();
17      for (String command : schema.split(";")) {
18        String sql = command.trim();
19        if (!sql.isEmpty()) {
20          statement.execute(sql);
21        }
22      }
23    }
24  }
25 }
```

Рисунок 2 – schema.sql

3) Инициализация БД при старте приложения (`App`)

Перед созданием окна JavaFx выполняется инициализация схемы. При ошибке приложение завершает работу с сообщением:

```

1 boolean authenticated = LoginDialog.showLoginDialog();
2 if (!authenticated) {
3     System.exit(0);
4 }
5
6 try {
7     DBConnection.initializeDatabase();
8     try (Connection conn = DBConnection.getConnection()) {
9         // Проверка подключения
10    }
11 } catch (SQLException | IOException ex) {
12     JOptionPane.showMessageDialog(null,
13         "Не удалось подключиться к БД: " + ex.getMessage(),
14         "Ошибка", JOptionPane.ERROR_MESSAGE);
15     System.exit(1);
16 }

```

Рисунок 3 – Инициализация схемы

4) SQL-схема с авто-созданием таблиц (`schema.sql`)

Схема хранится в `src/main/resources/db/schema.sql`. Для SQLite использован синтаксис `CREATE TABLE IF NOT EXISTS` и автогенерация ключей через `INTEGER`

`PRIMARY KEY AUTOINCREMENT`:

```

1 CREATE TABLE IF NOT EXISTS students (
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     first_name VARCHAR(50) NOT NULL,
4     last_name VARCHAR(50) NOT NULL,
5     group_name VARCHAR(20),
6     enrollment_date DATE
7 );
8
9 CREATE TABLE IF NOT EXISTS subjects (
10    id INTEGER PRIMARY KEY AUTOINCREMENT,
11    name VARCHAR(100) NOT NULL,
12    semester INT
13 );
14
15 CREATE TABLE IF NOT EXISTS grades (
16    id INTEGER PRIMARY KEY AUTOINCREMENT,
17    student_id INT REFERENCES students(id) ON DELETE CASCADE,
18    subject_id INT REFERENCES subjects(id) ON DELETE CASCADE,
19    grade INT CHECK (grade BETWEEN 1 AND 10),
20    date_assigned DATE DEFAULT CURRENT_DATE
21 );
22
23 CREATE INDEX IF NOT EXISTS idx_grades_student_id ON grades(student_id);
24 CREATE INDEX IF NOT EXISTS idx_grades_subject_id ON grades(subject_id);

```

Рисунок 4 – Автогенерация ключей

5) Добавление оценки и явная установка даты (`GradeDAO`)

Чтобы устранить ошибку при добавлении оценок, дата `date_assigned` задаётся явно при `INSERT`, а не полагается на автозаполнение:

```

1 public void insert(int studentId, int subjectId, int grade) throws SQLException {
2     String sql = "INSERT INTO grades(student_id, subject_id, grade, date_assigned) VALUES(?, ?, ?, ?)";
3     try (Connection conn = DBConnection.getConnection();
4         PreparedStatement ps = conn.prepareStatement(sql)) {
5         ps.setInt(1, studentId);
6         ps.setInt(2, subjectId);
7         ps.setInt(3, grade);
8         ps.setDate(4, Date.valueOf(LocalDate.now()));
9         ps.executeUpdate();
10    }
11 }

```

Рисунок 5 – Замена автозаполнения

6) Оформление пользовательского интерфейса (JavaFx)

Для повышения удобства использования были улучшены главное окно и панели с данными: добавлены отступы, заголовки для таблиц и единый шрифт.

6.1 Главное окно `MainFrame`

```
// Главное окно
FXMLLoader mainLoader = new FXMLLoader(getClass().getResource( name: "/FXML/MainView.fxml"));
Scene mainScene = new Scene(mainLoader.load());
mainScene.getStylesheets().add(getClass().getResource( name: "/css/styles.css").toExternalForm());

MainController mainController = mainLoader.getController();
mainController.setScene(mainScene);

primaryStage.setTitle("Контроль успеваемости студентов");
primaryStage.setScene(mainScene);
primaryStage.setWidth(900);
primaryStage.setHeight(600);
primaryStage.show();
```

Рисунок 6 – Инициализация Основного Окна

6.2 Панель студентов `StudentsPanel`

```
1 public StudentsPanel() {
2     setLayout(new BorderLayout(10, 10));
3     setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
4
5     model = new DefaultTableModel(new String[]{"ID", "Фамилия", "Имя", "Группа", "Дата"}, 0);
6     table = new JTable(model);
7     table.setFillViewportHeight(true);
8     JScrollPane scrollPane = new JScrollPane(table);
9     scrollPane.setBorder(BorderFactory.createTitledBorder("Список студентов"));
10    add(scrollPane, BorderLayout.CENTER);
11    ...
12 }
```

Рисунок 7 – Инициализация панели студентов

6.3 Панель оценок `GradesPanel`

```

@FXML new *
public void initialize() {
    System.out.println("GradesTabController initialized");

    // Настройка спиннера для оценок
    gradeSpinner.setValueFactory(new SpinnerValueFactory.IntegerSpinnerValueFactory( min: 1, max: 10, initialValue: 4));

    // Настройка колонок таблицы
    colId.setCellValueFactory( CellDataFeatures<Grade, Integer> cellData -> new SimpleIntegerProperty(cellData.getValue().getId()).asObject());
    colSubject.setCellValueFactory( CellDataFeatures<Grade, String> cellData -> new SimpleStringProperty(cellData.getValue().getSubject()));
    colGrade.setCellValueFactory( CellDataFeatures<Grade, Integer> cellData -> new SimpleIntegerProperty(cellData.getValue().getGrade()));
    colDate.setCellValueFactory( CellDataFeatures<Grade, String> cellData -> new SimpleStringProperty(cellData.getValue().getDateAssigned()));

    gradesTable.setItems(grades);

    // Загружаем данные в фоновом потоке
    Platform.runLater(() -> {
        loadStudents();
        loadSubjects();
    });
}

```

Рисунок 8 – Инициализация панели оценок

7) Темная тема, окно входа и навигация

Дополнительно реализованы:

- переключение светлой/темной темы в верхней панели;
- окно входа в аккаунт перед запуском основного интерфейса;
- навигационные кнопки по разделам приложения.

7.1 Включение темы в `App`

```

@FXML new *
private void setDarkTheme() {
    if (scene != null) {
        scene.getStylesheets().clear();
        scene.getStylesheets().add(getClass().getResource( name: "/css/styles.css").toExternalForm());
        isDark = true;
    }
}

@FXML new *
private void setLightTheme() {
    if (scene != null) {
        scene.getStylesheets().clear();
        scene.getStylesheets().add(getClass().getResource( name: "/css/light.css").toExternalForm());
        isDark = false;
    }
}

```

Рисунок 10 – Работа с темной темой

7.2 Окно входа в аккаунт (`LoginDialog`)

```

public class LoginController { 3 usages new *
    @FXML private TextField usernameField;
    @FXML private PasswordField passwordField;

    private static boolean authenticated = false; 3 usages

    @FXML new *
    private void handleLogin() {
        String username = usernameField.getText();
        String password = passwordField.getText();

        if ("admin".equals(username) && "1234".equals(password)) {
            authenticated = true;
            ((Stage) usernameField.getScene().getWindow()).close();
        } else {
            Alert alert = new Alert(Alert.AlertType.ERROR);
            alert.setTitle("Ошибка");
            alert.setHeaderText("Неверный логин или пароль");
            alert.showAndWait();
        }
    }
}

```

Рисунок 11 – Окно входа

Логин:

Пароль:

13

Вид: [Помощь](#)

Студенты [Предметы](#) [Оценки](#)

Студент: Предмет:

Оценка:

ID	Предмет	Оценка	Дата
1	Тимп	4	2026-05-28

Рисунок 16 – Интерфейс Оценок

ТЕСТИРОВАНИЕ

6.1. Цели и методы тестирования

На этапе разработки программного продукта критически важным этапом является верификация его корректности, надежности и устойчивости к нештатным ситуациям. Тестирование разработанной системы контроля успеваемости студентов проводилось с целью подтверждения соответствия функциональных возможностей приложения заявленным требованиям, а также проверки целостности данных в базе SQLite и корректности работы пользовательского интерфейса на базе JavaFX. В ходе работы применялись методы черного ящика (функциональное тестирование), при котором проверялось поведение системы без учета внутренней структуры кода, а также элементы белого ящика (структурное тестирование) для анализа обработки исключительных ситуаций в слое доступа к данным (DAO). Основной акцент был сделан на проверку граничных значений, валидацию пользовательского ввода и обеспечение ссылочной целостности между таблицами базы данных.

6.2. Тестирование модуля авторизации

Первым этапом тестирования стала проверка механизма ограничения доступа к системе. При запуске приложения пользователю предлагается окно авторизации (LoginView.fxml). В ходе серии экспериментов было установлено, что система корректно обрабатывает как успешные, так и неуспешные сценарии входа. При вводе корректных учетных данных (логин: admin, пароль: 1234) происходит закрытие окна авторизации и инициализация главного окна приложения (MainView.fxml), что подтверждает работоспособность логики переключения сцен в классе MainApp. В случае ввода неверного пароля или пустых полей система генерирует модальное окно ошибки типа Alert.AlertType.ERROR с информативным сообщением, при этом фокус остается на окне входа, и доступ к основным функциям программы блокируется. Также была проверена реакция системы на кнопку «Отмена»: приложение завершает свою работу через вызов System.exit(0), что предотвращает несанкционированный доступ к данным без прохождения процедуры аутентификации.

6.3. Тестирование модуля управления данными о студентах

Функциональность вкладки «Студенты» (StudentsTabController) подверглась тщательной проверке на предмет корректности операций CRUD (Create, Read, Update, Delete). При добавлении нового студента через форму ввода проверялась валидация данных. Система успешно принимает корректные данные, такие как фамилия, имя, номер группы и дата поступления в формате ГГГГ-ММ-ДД. Особое внимание уделялось обработке ошибочного ввода даты: при попытке ввести строку, не соответствующую формату LocalDate (например, «32.13.2025» или текстовые значения), приложение перехватывает исключение DateTimeParseException и выводит пользователю понятное сообщение об ошибке, не допуская падения программы или записи некорректных данных в базу.

Проверка операции чтения данных продемонстрировала, что таблица `TableView` корректно отображает список всех студентов из таблицы `students` базы данных сразу после загрузки вкладки. Механизм привязки данных через `ObservableList` обеспечивает автоматическое обновление интерфейса при изменении содержимого базы. Операция обновления существующей записи тестировалась путем изменения данных в полях ввода после выбора строки в таблице. Было подтверждено, что метод `update` в классе `StudentDAO` корректно формирует SQL-запрос с параметрами и обновляет запись по первичному ключу `id`. Удаление записей также работает штатно: при выборе студента и нажатии кнопки «Удалить» запись исчезает из интерфейса и удаляется из базы данных. Важным аспектом стало тестирование ссылочной целостности: при попытке удалить студента, имеющего связанные оценки в таблице `grades`, база данных SQLite (благодаря настройке `PRAGMA foreign_keys = ON` и ограничению `ON DELETE CASCADE`) либо каскадно удаляет связанные оценки, либо блокирует удаление (в зависимости от настроек схемы), что предотвращает появление «висячих» ссылок в базе данных.

6.4. Тестирование модуля управления учебными дисциплинами

Аналогичные тесты были проведены для вкладки «Предметы» (`SubjectsTabController`). Проверка добавления новых дисциплин показала, что система корректно обрабатывает название предмета и номер семестра. Использование элемента управления `Spinner` для выбора семестра ограничивает ввод пользователя диапазоном от 1 до 8, что исключает возможность внесения абсурдных данных (например, отрицательных номеров семестров или чисел больше 12). Тестирование редактирования подтвердило, что изменения названия дисциплины сохраняются в базе данных и немедленно отражаются в таблице интерфейса. Удаление предметов также было протестировано на наличие связанных оценок. В соответствии со схемой базы данных (`schema.sql`), удаление предмета приводит к каскадному удалению всех связанных с ним оценок из таблицы `grades`, что было успешно подтверждено в ходе экспериментов: после удаления предмета «Высшая математика» все оценки по этому предмету исчезали из журнала успеваемости соответствующих студентов.

6.5. Тестирование модуля журнала оценок

Наиболее сложным с точки зрения логики является модуль «Оценки» (`GradesTabController`), так как он агрегирует данные из трех сущностей: студентов, предметов и самих оценок. Тестирование началось с проверки загрузки выпадающих списков (`ComboBox`). При открытии вкладки система автоматически загружает списки всех зарегистрированных студентов и предметов. Было выявлено, что при отсутствии данных в базах `students` или `subjects` списки остаются пустыми, а интерфейс не падает с ошибкой `NullPointerException`, благодаря наличию проверок на `null` в обработчиках событий.

Основной сценарий тестирования включал добавление оценки выбранному студенту по выбранному предмету. Система корректно считывает значение из `Spinner` оценок (диапазон 1–10) и текущую дату. При нажатии кнопки «Добавить оценка» вызывается метод `insert` класса `GradeDAO`, который явно устанавливает дату выставления оценки через `LocalDate.now()`, что решает проблему возможного рассогласования времени сервера и клиента. После успешного добавления таблица оценок обновляется, отображая новый ряд с названием предмета, баллом и датой.

Также была проверена фильтрация оценок по студенту. При смене выбранного студента в верхнем выпадающем списке таблица оценок очищается и заполняется заново данными только для нового выбранного обучающегося. Это подтверждает корректную работу SQL-запроса с условием `WHERE s.id = ?` в методе `findByStudent` класса `GradeDAO`. Тестирование граничных значений показало, что система не позволяет добавить оценку вне диапазона 1–10 благодаря ограничениям интерфейса (`SpinnerValueFactory`) и проверкам на уровне базы данных (`CHECK (grade BETWEEN 1 AND 10)`).

6.6. Тестирование пользовательского интерфейса и стилей

Отдельное внимание было уделено визуальной части приложения. Проверка переключения тем оформления («Тёмная тема» / «Светлая тема») через меню «Вид» показала, что стили CSS (`styles.css` и `light.css`) применяются динамически без необходимости перезапуска приложения. Цветовая схема элементов управления, таких как таблицы, кнопки и текстовые поля, изменяется корректно, обеспечивая достаточный контраст для комфортного чтения информации. Адаптивность интерфейса тестировалась путем изменения размеров окна приложения: элементы `VBox` и `GridPane` корректно перераспределяют пространство, а таблицы растягиваются на всю доступную область, используя свойство `VBox.vgrow="ALWAYS"`.

6.7. Тестирование устойчивости к ошибкам и исключительным ситуациям

Для проверки надежности системы моделировались различные аварийные ситуации. Во-первых, была проверена работа приложения при отсутствии файла базы данных `students.db`. При первом запуске файл отсутствует, однако метод `initializeDatabase` в классе `DBConnection` успешно создает его и выполняет скрипт `schema.sql`, создавая необходимые таблицы. Во-вторых, тестировалась обработка ошибок подключения к базе данных. Хотя в локальном режиме с `SQLite` ошибки подключения маловероятны, код обернут в блоки `try-catch`, которые перехватывают `SQLException` и выводят пользователю диагностические сообщения через стандартные диалоговые окна `JavaFX`, предотвращая аварийное завершение процесса JVM.

Также проверялась устойчивость к некорректному вводу числовых данных. Поскольку поля для ввода ID или оценок используют специализированные контролы (`Spinner`, `ComboBox`) или преобразуются внутри кода с

обработкой исключений, риск ввода букв вместо цифр сведен к минимуму. В тех случаях, где используется прямой ввод текста (например, дата), реализована строгая валидация формата, и любые отклонения от стандарта ISO 8601 блокируются на уровне контроллера.

6.8. Результаты тестирования

По итогам проведенного комплекса испытаний можно констатировать, что разработанное программное обеспечение «Контроль успеваемости студентов» функционирует стабильно и соответствует всем поставленным требованиям. Все основные функциональные модули (авторизация, управление студентами, предметами и оценками) работают корректно. Данные сохраняются в базе данных SQLite без потерь и искажений, ссылочная целостность соблюдается благодаря механизмам внешнего ключа и каскадного удаления. Пользовательский интерфейс отзывчив, информативен и защищает пользователя от совершения логических ошибок ввода. Выявленные в ходе начальных этапов разработки недочеты (такие как необходимость явного указания даты при добавлении оценки) были успешно устранены, что подтверждается повторными тестами. Таким образом, приложение готово к практической эксплуатации в качестве учебного инструмента или прототипа информационной системы учебного заведения.

ВЫВОД

В ходе выполнения данной курсовой работы была разработана система контроля успеваемости студентов, реализованная на языке программирования

Java версии 23 с использованием встраиваемой системы управления базами данных SQLite. Выбор SQLite в качестве средства хранения данных обусловлен его очевидными преимуществами для подобного класса приложений: отсутствие необходимости в развёртывании и администрировании отдельного сервера базы данных, компактность, высокая скорость работы и простота интеграции с Java-приложением через встроенный JDBC-драйвер. Это позволило создать полностью автономное решение, которое может функционировать на любом компьютере без дополнительных настроек и установки стороннего программного обеспечения.

Разработанное приложение предоставляет пользователю полный набор функций, необходимых для ведения учёта успеваемости в учебном заведении. В первую очередь, реализован модуль управления данными о студентах, который обеспечивает возможность добавления новых обучающихся в базу данных, редактирования их личной информации (фамилия, имя, отчество, номер студенческого билета, академическая группа, дата поступления), а также удаления записей о студентах при отчислении или завершении обучения. Все операции выполняются с контролем корректности вводимых данных и выводом соответствующих информационных сообщений для пользователя.

Во-вторых, в системе реализован модуль управления перечнем учебных предметов. Преподаватель или сотрудник учебного отдела имеет возможность добавлять новые дисциплины, указывая их название, семестр изучения, количество академических часов, а также форму промежуточной аттестации. Предусмотрена функция редактирования информации о существующих предметах и удаления дисциплин, которые более не входят в учебный план.

В-третьих, ключевым функциональным элементом системы является журнал оценок. Данный модуль позволяет выставять студентам оценки по различным дисциплинам с обязательной фиксацией даты выставления. Реализована возможность редактирования ранее выставленных оценок в случае обнаружения ошибки или пересдачи экзамена. Все оценки надёжно

привязаны к конкретным студентам и конкретным предметам, что обеспечивает целостность и непротиворечивость данных на уровне ссылочной целостности базы данных SQLite.

В-четвёртых, система предоставляет удобный механизм просмотра успеваемости. Пользователь может получить полный список всех оценок конкретного студента с указанием по каждому предмету выставленной оценки и даты её получения. Это позволяет быстро оценить текущую академическую успеваемость обучающегося, выявить проблемные дисциплины и своевременно принять необходимые меры.

Все перечисленные функции объединены в единое приложение с текстовым консольным интерфейсом, построенным по принципу иерархического меню. Пользователю достаточно вводить цифровые команды для навигации по разделам системы и выполнения операций. Интерфейс выполнен на русском языке, содержит понятные подсказки и информативные сообщения об успешном выполнении операций или возникших ошибках.

Архитектура приложения построена по принципу разделения ответственности: выделены отдельные классы для работы с базой данных (DAO-слой), классы, реализующие бизнес-логику (сервисный слой), и классы, отвечающие за взаимодействие с пользователем (слой представления). Такой подход обеспечивает модульность кода, упрощает его сопровождение и тестирование, а также создаёт основу для возможного расширения функциональности в будущем, например, добавления графического интерфейса или экспорта отчётов в распространённые форматы.

Таким образом, все поставленные в работе задачи успешно решены. Разработанная система контроля успеваемости студентов представляет собой готовое к практическому использованию программное решение, которое может быть внедрено в учебном процессе для автоматизации учёта и анализа академической успеваемости обучающихся. Система отличается простотой использования, автономностью и достаточным для повседневной работы набором функций.

ПРИЛОЖЕНИЕ

```

/dao
DBConnection
package com.example.studentcontrol.dao;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.sql.*;

public class DBConnection {
    private static final String URL =
"jdbc:sqlite:students.db?date_string_format=yyyy-MM-dd";

    public static Connection getConnection() throws SQLException {
        Connection conn = DriverManager.getConnection(URL);
        try (Statement stmt = conn.createStatement()) {
            stmt.execute("PRAGMA foreign_keys = ON");
        }
        return conn;
    }

    public static void initializeDatabase() throws SQLException, IOException {
        try (Connection conn = getConnection(); Statement stmt =
conn.createStatement()) {
            String schema = readSchema();
            for (String command : schema.split(";")) {
                String sql = command.trim();
                if (!sql.isEmpty()) {
                    stmt.execute(sql);
                }
            }
        }
    }

    private static String readSchema() throws IOException {
        try (InputStream is =
DBConnection.class.getResourceAsStream("/db/schema.sql")) {
            if (is == null) throw new IOException("schema.sql not found");
            ByteArrayOutputStream os = new ByteArrayOutputStream();
            byte[] buffer = new byte[4096];
            int len;
            while ((len = is.read(buffer)) != -1) {
                os.write(buffer, 0, len);
            }
            return os.toString(StandardCharsets.UTF_8);
        }
    }
}

```



```
    }  
}
```

GradeDAO

```
package com.example.studentcontrol.dao;
```

```
import com.example.studentcontrol.model.Grade;  
import com.example.studentcontrol.model.Student;  
import com.example.studentcontrol.model.Subject;  
import java.sql.*;  
import java.time.LocalDate;  
import java.util.ArrayList;  
import java.util.List;
```

```
public class GradeDAO {  
    public List<Grade> findByStudent(int studentId) throws SQLException {  
        List<Grade> list = new ArrayList<>();  
        String sql = "SELECT g.id, g.grade, g.date_assigned, " +  
            "s.id AS sid, s.first_name, s.last_name, s.group_name, s.enrollment_date,  
" +  
            "sub.id AS subid, sub.name, sub.semester " +  
            "FROM grades g " +  
            "JOIN students s ON g.student_id = s.id " +  
            "JOIN subjects sub ON g.subject_id = sub.id " +  
            "WHERE s.id = ?";  
        try (Connection conn = DBConnection.getConnection();  
            PreparedStatement ps = conn.prepareStatement(sql)) {  
            ps.setInt(1, studentId);  
            try (ResultSet rs = ps.executeQuery()) {  
                while (rs.next()) {  
                    Student st = new Student(  
                        rs.getInt("sid"),  
                        rs.getString("first_name"),  
                        rs.getString("last_name"),  
                        rs.getString("group_name"),  
                        rs.getDate("enrollment_date").toLocalDate()  
                    );  
                    Subject sub = new Subject(  
                        rs.getInt("subid"),  
                        rs.getString("name"),  
                        rs.getInt("semester")  
                    );  
                    Grade gr = new Grade(  
                        rs.getInt("id"),  
                        st,  

```

```

        sub,
        rs.getInt("grade"),
        rs.getDate("date_assigned").toLocalDate()
    );
    list.add(gr);
}
}
}
return list;
}

public void insert(int studentId, int subjectId, int grade) throws SQLException {
    String sql = "INSERT INTO grades(student_id, subject_id, grade,
date_assigned) VALUES(?,?,?,?)";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, studentId);
        ps.setInt(2, subjectId);
        ps.setInt(3, grade);
        ps.setDate(4, Date.valueOf(LocalDate.now()));
        ps.executeUpdate();
    }
}

public void delete(int id) throws SQLException {
    String sql = "DELETE FROM grades WHERE id=?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        ps.executeUpdate();
    }
}
}
}

```

StudentDAO

```
package com.example.studentcontrol.dao;
```

```
import com.example.studentcontrol.model.Student;
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
```

```
public class StudentDAO {
    public List<Student> findAll() throws SQLException {
```

```

List<Student> list = new ArrayList<>();
String sql = "SELECT * FROM students";
try (Connection conn = DBConnection.getConnection();
    Statement st = conn.createStatement();
    ResultSet rs = st.executeQuery(sql)) {
    while (rs.next()) {
        Student s = new Student(
            rs.getInt("id"),
            rs.getString("first_name"),
            rs.getString("last_name"),
            rs.getString("group_name"),
            rs.getDate("enrollment_date").toLocalDate());
        list.add(s);
    }
}
return list;
}

```

```

public void insert(Student s) throws SQLException {
    String sql = "INSERT INTO students(first_name, last_name, group_name,
enrollment_date) VALUES(?,?,?,?)";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, s.getFirstName());
        ps.setString(2, s.getLastName());
        ps.setString(3, s.getGroupName());
        ps.setDate(4, Date.valueOf(s.getEnrollmentDate()));
        ps.executeUpdate();
    }
}

```

```

public void update(Student s) throws SQLException {
    String sql = "UPDATE students SET first_name=?, last_name=?,
group_name=?, enrollment_date=? WHERE id=?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, s.getFirstName());
        ps.setString(2, s.getLastName());
        ps.setString(3, s.getGroupName());
        ps.setDate(4, Date.valueOf(s.getEnrollmentDate()));
        ps.setInt(5, s.getId());
        ps.executeUpdate();
    }
}

```

```

public void delete(int id) throws SQLException {
    String sql = "DELETE FROM students WHERE id = ?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        ps.executeUpdate();
    }
}
}
}

```

SubjectDAO

```
package com.example.studentcontrol.dao;
```

```

import com.example.studentcontrol.model.Subject;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

```

```

public class SubjectDAO {
    public List<Subject> findAll() throws SQLException {
        List<Subject> list = new ArrayList<>();
        String sql = "SELECT * FROM subjects";
        try (Connection conn = DBConnection.getConnection();
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                Subject s = new Subject(
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getInt("semester"));
                list.add(s);
            }
        }
        return list;
    }

    public void insert(Subject s) throws SQLException {
        String sql = "INSERT INTO subjects(name, semester) VALUES(?,?)";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, s.getName());
            ps.setInt(2, s.getSemester());
            ps.executeUpdate();
        }
    }
}

```

```

    }

    public void update(Subject s) throws SQLException {
        String sql = "UPDATE subjects SET name=?, semester=? WHERE id=?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, s.getName());
            ps.setInt(2, s.getSemester());
            ps.setInt(3, s.getId());
            ps.executeUpdate();
        }
    }

    public void delete(int id) throws SQLException {
        String sql = "DELETE FROM subjects WHERE id=?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}

/model

Grade
package com.example.studentcontrol.model;

import java.time.LocalDate;

public class Grade {
    private int id;
    private Student student;
    private Subject subject;
    private int grade;
    private LocalDate dateAssigned;

    public Grade() {}

    public Grade(int id, Student student, Subject subject, int grade, LocalDate
dateAssigned) {
        this.id = id;
        this.student = student;
        this.subject = subject;
        this.grade = grade;
    }
}

```

```

        this.dateAssigned = dateAssigned;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public Student getStudent() { return student; }
    public void setStudent(Student student) { this.student = student; }
    public Subject getSubject() { return subject; }
    public void setSubject(Subject subject) { this.subject = subject; }
    public int getGrade() { return grade; }
    public void setGrade(int grade) { this.grade = grade; }
    public LocalDate getDateAssigned() { return dateAssigned; }
    public void setDateAssigned(LocalDate dateAssigned) { this.dateAssigned =
dateAssigned; }
}

```

Student

```
package com.example.studentcontrol.model;
```

```
import java.time.LocalDate;
```

```

public class Student {
    private int id;
    private String firstName;
    private String lastName;
    private String groupName;
    private LocalDate enrollmentDate;

    public Student() {}

    public Student(int id, String firstName, String lastName, String groupName,
LocalDate enrollmentDate) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.groupName = groupName;
        this.enrollmentDate = enrollmentDate;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
}

```

```

    public String getGroupName() { return groupName; }
    public void setGroupName(String groupName) { this.groupName =
groupName; }
    public LocalDate getEnrollmentDate() { return enrollmentDate; }
    public void setEnrollmentDate(LocalDate enrollmentDate)
{ this.enrollmentDate = enrollmentDate; }

    @Override
    public String toString() {
        return lastName + " " + firstName + " (" + groupName + ")";
    }
}

```

Subject

```
package com.example.studentcontrol.model;
```

```

public class Subject {
    private int id;
    private String name;
    private int semester;

    public Subject() {}

    public Subject(int id, String name, int semester) {
        this.id = id;
        this.name = name;
        this.semester = semester;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getSemester() { return semester; }
    public void setSemester(int semester) { this.semester = semester; }

    @Override
    public String toString() {
        return name + " (sem. " + semester + ")";
    }
}

```

UI/

GradesTabController

```
package com.example.studentcontrol.ui;

import com.example.studentcontrol.dao.GradeDAO;
import com.example.studentcontrol.dao.StudentDAO;
import com.example.studentcontrol.dao.SubjectDAO;
import com.example.studentcontrol.model.Grade;
import com.example.studentcontrol.model.Student;
import com.example.studentcontrol.model.Subject;
import javafx.application.Platform;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;

import java.sql.SQLException;
import java.util.List;

public class GradesTabController {
    @FXML private ComboBox<Student> studentCombo;
    @FXML private ComboBox<Subject> subjectCombo;
    @FXML private Spinner<Integer> gradeSpinner;

    @FXML private TableView<Grade> gradesTable;
    @FXML private TableColumn<Grade, Integer> colId;
    @FXML private TableColumn<Grade, String> colSubject;
    @FXML private TableColumn<Grade, Integer> colGrade;
    @FXML private TableColumn<Grade, String> colDate;

    private StudentDAO studentDAO = new StudentDAO();
    private SubjectDAO subjectDAO = new SubjectDAO();
    private GradeDAO gradeDAO = new GradeDAO();
    private ObservableList<Grade> grades = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        System.out.println("GradesTabController initialized");

        // Настройка спиннера для оценок
        gradeSpinner.setValueFactory(new
SpinnerValueFactory.IntegerSpinnerValueFactory(1, 10, 4));

        // Настройка колонок таблицы
```



```

        colId.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().getId()).asObject());
        colSubject.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getSubject().getName()));
        colGrade.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().getGrade()).asObject());
        colDate.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getDateAssigned().toString()));

        gradesTable.setItems(grades);

        // Загружаем данные в фоновом потоке
        Platform.runLater(() -> {
            loadStudents();
            loadSubjects();
        });

        // Слушатель выбора студента
        studentCombo.getSelectionModel().selectedItemProperty().addListener((obs,
oldVal, newVal) -> {
            if (newVal != null) {
                System.out.println("Selected student: " + newVal.getLastName());
                loadGrades(newVal.getId());
            }
        });
    }

    private void loadStudents() {
        try {
            List<Student> students = studentDAO.findAll();
            System.out.println("Loaded " + students.size() + " students");
            ObservableList<Student> studentList =
FXCollections.observableArrayList(students);
            studentCombo.setItems(studentList);

            // Если есть студенты, выбираем первого
            if (!studentList.isEmpty()) {
                studentCombo.getSelectionModel().selectFirst();
            }
        } catch (SQLException e) {
            System.err.println("Error loading students: " + e.getMessage());
            e.printStackTrace();
            showError("Ошибка загрузки студентов: " + e.getMessage());
        }
    }
}

```

```

private void loadSubjects() {
    try {
        List<Subject> subjects = subjectDAO.findAll();
        System.out.println("Loaded " + subjects.size() + " subjects");
        ObservableList<Subject> subjectList =
FXCollections.observableArrayList(subjects);
        subjectCombo.setItems(subjectList);
    } catch (SQLException e) {
        System.err.println("Error loading subjects: " + e.getMessage());
        e.printStackTrace();
        showError("Ошибка загрузки предметов: " + e.getMessage());
    }
}

private void loadGrades(int studentId) {
    try {
        List<Grade> gradeList = gradeDAO.findByStudent(studentId);
        System.out.println("Loaded " + gradeList.size() + " grades for student " +
studentId);
        grades.setAll(gradeList);
    } catch (SQLException e) {
        System.err.println("Error loading grades: " + e.getMessage());
        e.printStackTrace();
        showError("Ошибка загрузки оценок: " + e.getMessage());
    }
}

@FXML
private void handleAdd() {
    Student student = studentCombo.getSelectionModel().getSelectedItem();
    Subject subject = subjectCombo.getSelectionModel().getSelectedItem();

    if (student == null) {
        showError("Выберите студента");
        return;
    }

    if (subject == null) {
        showError("Выберите предмет");
        return;
    }

    int grade = gradeSpinner.getValue();

```

```

try {
    gradeDAO.insert(student.getId(), subject.getId(), grade);
    loadGrades(student.getId());

    // Показываем сообщение об успехе
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("Успех");
    alert.setHeaderText(null);
    alert.setContentText("Оценка " + grade + " добавлена студенту " +
student.getLastName());
    alert.showAndWait();

} catch (SQLException e) {
    System.err.println("Error adding grade: " + e.getMessage());
    e.printStackTrace();
    showError("Ошибка добавления оценки: " + e.getMessage());
}
}

private void showError(String msg) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Ошибка");
    alert.setHeaderText(null);
    alert.setContentText(msg);
    alert.showAndWait();
}
}

```

loginControler

```
package com.example.studentcontrol.ui;
```

```
import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.stage.Stage;
```

```
public class LoginController {
    @FXML private TextField usernameField;
    @FXML private PasswordField passwordField;

    private static boolean authenticated = false;

    @FXML
    private void handleLogin() {
        String username = usernameField.getText();
        String password = passwordField.getText();
    }
}

```

```

        if ("admin".equals(username) && "1234".equals(password)) {
            authenticated = true;
            ((Stage) usernameField.getScene().getWindow()).close();
        } else {
            Alert alert = new Alert(Alert.AlertType.ERROR);
            alert.setTitle("Ошибка");
            alert.setHeaderText("Неверный логин или пароль");
            alert.showAndWait();
        }
    }

    @FXML
    private void handleCancel() {
        authenticated = false;
        ((Stage) usernameField.getScene().getWindow()).close();
    }

    public static boolean isAuthenticated() {
        return authenticated;
    }
}

```

```

mainController
package com.example.studentcontrol.ui;

```

```

import javafx.fxml.FXML;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.TabPane;
import javafx.stage.Stage;

public class MainController {
    @FXML private TabPane tabPane;
    private Scene scene;
    private boolean isDark = true;

    public void setScene(Scene scene) {
        this.scene = scene;
    }

    @FXML
    private void setDarkTheme() {
        if (scene != null) {
            scene.getStylesheets().clear();

```

```
scene.getStylesheets().add(getClass().getResource("/css/styles.css").toExternalForm());
```

```
    isDark = true;
  }
}
```

```
@FXML
```

```
private void setLightTheme() {
    if (scene != null) {
        scene.getStylesheets().clear();
```

```
scene.getStylesheets().add(getClass().getResource("/css/light.css").toExternalForm());
```

```
    isDark = false;
  }
}
```

```
@FXML
```

```
private void showAbout() {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("О программе");
    alert.setHeaderText("Контроль успеваемости студентов");
    alert.setContentText("Версия 1.0\nРазработано с использованием JavaFX и SQLite");
    alert.showAndWait();
}
}
```

```
StudentsTabControler
```

```
package com.example.studentcontrol.ui;
```

```
import com.example.studentcontrol.dao.StudentDAO;
import com.example.studentcontrol.model.Student;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleObjectProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;
```

```
import java.time.LocalDate;
import java.util.List;
```

```

public class StudentsTabController {
    @FXML private TableView<Student> studentsTable;
    @FXML private TableColumn<Student, Integer> colId;
    @FXML private TableColumn<Student, String> colLastName;
    @FXML private TableColumn<Student, String> colFirstName;
    @FXML private TableColumn<Student, String> colGroup;
    @FXML private TableColumn<Student, LocalDate> colDate;

    @FXML private TextField lastNameField;
    @FXML private TextField firstNameField;
    @FXML private TextField groupField;
    @FXML private TextField dateField;

    private StudentDAO studentDAO = new StudentDAO();
    private ObservableList<Student> students =
FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        colId.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().getId()).asObject());
        colLastName.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getLastName()));
        colFirstName.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getFirstName()));
        colGroup.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getGroupName()));
        colDate.setCellValueFactory(cellData -> new
SimpleObjectProperty<>(cellData.getValue().getEnrollmentDate()));

        studentsTable.setItems(students);
        refreshTable();
    }

    @FXML
    private void handleAdd() {
        try {
            Student s = new Student();
            s.setFirstName(firstNameField.getText());
            s.setLastName(lastNameField.getText());
            s.setGroupName(groupField.getText());
            s.setEnrollmentDate(LocalDate.parse(dateField.getText()));
            studentDAO.insert(s);
            refreshTable();
            clearFields();
        }
    }
}

```

```

    } catch (Exception e) {
        showError("Ошибка добавления: " + e.getMessage());
    }
}

```

@FXML

```

private void handleUpdate() {
    Student selected = studentsTable.getSelectionModel().getSelectedItem();
    if (selected == null) {
        showError("Выберите студента для обновления");
        return;
    }
    try {
        selected.setFirstName(firstNameField.getText());
        selected.setLastName(lastNameField.getText());
        selected.setGroupName(groupField.getText());
        selected.setEnrollmentDate(LocalDate.parse(dateField.getText()));
        studentDAO.update(selected);
        refreshTable();
        clearFields();
    } catch (Exception e) {
        showError("Ошибка обновления: " + e.getMessage());
    }
}

```

@FXML

```

private void handleDelete() {
    Student selected = studentsTable.getSelectionModel().getSelectedItem();
    if (selected == null) {
        showError("Выберите студента для удаления");
        return;
    }
    try {
        studentDAO.delete(selected.getId());
        refreshTable();
    } catch (Exception e) {
        showError("Ошибка удаления: " + e.getMessage());
    }
}

```

```

private void refreshTable() {
    try {
        List<Student> list = studentDAO.findAll();
        students.setAll(list);
    } catch (Exception e) {

```

```

        showError("Ошибка загрузки: " + e.getMessage());
    }
}

private void clearFields() {
    lastNameField.clear();
    firstNameField.clear();
    groupField.clear();
    dateField.clear();
}

private void showError(String msg) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Ошибка");
    alert.setContentText(msg);
    alert.showAndWait();
}
}

```

SubjectsTabController

```
package com.example.studentcontrol.ui;
```

```

import com.example.studentcontrol.dao.SubjectDAO;
import com.example.studentcontrol.model.Subject;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;

```

```
import java.util.List;
```

```

public class SubjectsTabController {
    @FXML private TableView<Subject> subjectsTable;
    @FXML private TableColumn<Subject, Integer> colId;
    @FXML private TableColumn<Subject, String> colName;
    @FXML private TableColumn<Subject, Integer> colSemester;

    @FXML private TextField nameField;
    @FXML private Spinner<Integer> semesterSpinner;

    private SubjectDAO subjectDAO = new SubjectDAO();
    private ObservableList<Subject> subjects =
FXCollections.observableArrayList();

```



```

@FXML
public void initialize() {
    colId.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().getId()).asObject());
    colName.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().getName()));
    colSemester.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().getSemester()).asObject());

    semesterSpinner.setValueFactory(new
SpinnerValueFactory.IntegerSpinnerValueFactory(1, 8, 1));

    subjectsTable.setItems(subjects);
    refreshTable();
}

```

```

@FXML
private void handleAdd() {
    try {
        Subject s = new Subject();
        s.setName(nameField.getText());
        s.setSemester(semesterSpinner.getValue());
        subjectDAO.insert(s);
        refreshTable();
        clearFields();
    } catch (Exception e) {
        showError("Ошибка добавления: " + e.getMessage());
    }
}

```

```

@FXML
private void handleUpdate() {
    Subject selected = subjectsTable.getSelectionModel().getSelectedItem();
    if (selected == null) {
        showError("Выберите предмет для обновления");
        return;
    }
    try {
        selected.setName(nameField.getText());
        selected.setSemester(semesterSpinner.getValue());
        subjectDAO.update(selected);
        refreshTable();
        clearFields();
    } catch (Exception e) {
        showError("Ошибка обновления: " + e.getMessage());
    }
}

```

```

    }
}

@FXML
private void handleDelete() {
    Subject selected = subjectsTable.getSelectionModel().getSelectedItem();
    if (selected == null) {
        showError("Выберите предмет для удаления");
        return;
    }
    try {
        subjectDAO.delete(selected.getId());
        refreshTable();
    } catch (Exception e) {
        showError("Ошибка удаления: " + e.getMessage());
    }
}

private void refreshTable() {
    try {
        List<Subject> list = subjectDAO.findAll();
        subjects.setAll(list);
    } catch (Exception e) {
        showError("Ошибка загрузки: " + e.getMessage());
    }
}

private void clearFields() {
    nameField.clear();
    semesterSpinner.getValueFactory().setValue(1);
}

private void showError(String msg) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Ошибка");
    alert.setContentText(msg);
    alert.showAndWait();
}
}

/MainApp
package com.example.studentcontrol;

import com.example.studentcontrol.dao.DBConnection;
import com.example.studentcontrol.ui.LoginController;
import com.example.studentcontrol.ui.MainController;
import javafx.application.Application;

```

```

import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.stage.Stage;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        // Инициализация БД
        try {
            DBConnection.initializeDatabase();
        } catch (Exception e) {
            Alert alert = new Alert(Alert.AlertType.ERROR);
            alert.setTitle("Ошибка");
            alert.setHeaderText("Не удалось инициализировать БД");
            alert.setContentText(e.getMessage());
            alert.showAndWait();
            System.exit(1);
        }

        // Показываем окно входа
        FXMLLoader loginLoader = new
        FXMLLoader(getClass().getResource("/fxml/LoginView.fxml"));
        Scene loginScene = new Scene(loginLoader.load());
        Stage loginStage = new Stage();
        loginStage.setTitle("Авторизация");
        loginStage.setScene(loginScene);
        loginStage.setResizable(false);
        loginStage.showAndWait();

        if (!LoginController.isAuthenticated()) {
            System.exit(0);
        }

        // Главное окно
        FXMLLoader mainLoader = new
        FXMLLoader(getClass().getResource("/fxml/MainView.fxml"));
        Scene mainScene = new Scene(mainLoader.load());

        mainScene.getStylesheets().add(getClass().getResource("/css/styles.css").toExternalForm());

        MainController mainController = mainLoader.getController();
        mainController.setScene(mainScene);

        primaryStage.setTitle("Контроль успеваемости студентов");
        primaryStage.setScene(mainScene);
        primaryStage.setWidth(900);
        primaryStage.setHeight(600);
        primaryStage.show();
    }
}

```

```

    public static void main(String[] args) {
        launch(args);
    }
}

/resources

/css
light.css
.root {
    -fx-base: #f0f0f0;
    -fx-background: #f5f5f5;
    -fx-control-inner-background: white;
    -fx-text-fill: #333333;
}

.tab-pane {
    -fx-background-color: #f5f5f5;
}

.tab {
    -fx-background-color: #e0e0e0;
}

.tab:selected {
    -fx-background-color: #4a6ea9;
}

.table-view {
    -fx-background-color: white;
}

.button {
    -fx-background-color: #4a6ea9;
    -fx-text-fill: white;
}

.label {
    -fx-text-fill: #333333;
}
styles.css
.root {
    -fx-base: #2b2b2b;
    -fx-background: #3c3f41;
    -fx-control-inner-background: #4a4c4e;
    -fx-text-fill: #e0e0e0;
}

```

```

    -fx-font-family: "Segoe UI";
    -fx-font-size: 13px;
}

.tab-pane {
    -fx-background-color: #3c3f41;
}

.tab {
    -fx-background-color: #2b2b2b;
    -fx-text-fill: #cccccc;
}

.tab:selected {
    -fx-background-color: #4a6ea9;
}

.tab .tab-label {
    -fx-text-fill: white;
}

.table-view {
    -fx-background-color: #4a4c4e;
    -fx-border-color: #6e6e6e;
}

.table-cell {
    -fx-text-fill: white;
}

.table-row-cell:odd {
    -fx-background-color: #555758;
}

.table-row-cell:selected {
    -fx-background-color: #4a6ea9;
}

.button {
    -fx-background-color: #5a5d63;
    -fx-text-fill: white;
    -fx-background-radius: 8;
    -fx-padding: 8 16;
}

.button:hover {

```

```
-fx-background-color: #6e737b;  
}
```

```
.text-field, .combo-box, .spinner {  
  -fx-background-color: #555758;  
  -fx-text-fill: white;  
  -fx-background-radius: 6;  
}
```

```
.label {  
  -fx-text-fill: #dddddd;  
}
```

```
.menu-bar {  
  -fx-background-color: #2b2b2b;  
}
```

```
.menu {  
  -fx-text-fill: white;  
}
```

```
.dialog-pane {  
  -fx-background-color: #3c3f41;  
}
```

```
.dialog-pane .label {  
  -fx-text-fill: white;  
}
```

/db

schema.sql

```
CREATE TABLE IF NOT EXISTS students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  group_name VARCHAR(20),  
  enrollment_date DATE  
);
```

```
CREATE TABLE IF NOT EXISTS subjects (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name VARCHAR(100) NOT NULL,  
  semester INT  
);
```

```
CREATE TABLE IF NOT EXISTS grades (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        student_id INT REFERENCES students(id) ON DELETE CASCADE,
        subject_id INT REFERENCES subjects(id) ON DELETE CASCADE,
        grade INT CHECK (grade BETWEEN 1 AND 10),
        date_assigned DATE DEFAULT CURRENT_DATE
    );

CREATE INDEX IF NOT EXISTS idx_grades_student_id ON grades(student_id);
CREATE INDEX IF NOT EXISTS idx_grades_subject_id ON grades(subject_id);

```

```

/fxml
GradesTab.fxml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox spacing="10" xmlns="http://javafx.com/javafx/21"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.example.studentcontrol.ui.GradesTabController">
    <padding>
        <Insets top="10" right="10" bottom="10" left="10"/>
    </padding>

    <!-- Панель выбора студента и предмета -->
    <GridPane hgap="10" vgap="10">
        <padding>
            <Insets bottom="10"/>
        </padding>

        <Label text="Студент:" GridPane.rowIndex="0"
GridPane.columnIndex="0"/>
        <ComboBox fx:id="studentCombo" GridPane.rowIndex="0"
GridPane.columnIndex="1" prefWidth="250"/>

        <Label text="Предмет:" GridPane.rowIndex="0"
GridPane.columnIndex="2"/>
        <ComboBox fx:id="subjectCombo" GridPane.rowIndex="0"
GridPane.columnIndex="3" prefWidth="250"/>

        <Label text="Оценка:" GridPane.rowIndex="1"
GridPane.columnIndex="0"/>
        <Spinner fx:id="gradeSpinner" GridPane.rowIndex="1"
GridPane.columnIndex="1" prefWidth="80"/>
    </GridPane>
</VBox>

```

```

        <Button text="Добавить оценку" onAction="#handleAdd"
GridPane.rowIndex="1" GridPane.columnIndex="3"/>
    </GridPane>

    <!-- Таблица с оценками -->
    <TableView fx:id="gradesTable" VBox.vgrow="ALWAYS">
        <columns>
            <TableColumn text="ID" prefWidth="50" fx:id="colId"/>
            <TableColumn text="Предмет" prefWidth="300" fx:id="colSubject"/>
            <TableColumn text="Оценка" prefWidth="80" fx:id="colGrade"/>
            <TableColumn text="Дата" prefWidth="120" fx:id="colDate"/>
        </columns>
        <placeholder>
            <Label text="Нет оценок у выбранного студента"/>
        </placeholder>
    </TableView>
</VBox>

loginView.fxml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<GridPane alignment="CENTER" hgap="10" vgap="10"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.example.studentcontrol.ui.LoginController">
    <padding>
        <Insets top="20" right="20" bottom="20" left="20"/>
    </padding>

    <Label text="Логин:" GridPane.rowIndex="0" GridPane.columnIndex="0"/>
    <TextField fx:id="usernameField" prefWidth="200" GridPane.rowIndex="0"
GridPane.columnIndex="1"/>

    <Label text="Пароль:" GridPane.rowIndex="1" GridPane.columnIndex="0"/>
    <PasswordField fx:id="passwordField" prefWidth="200"
GridPane.rowIndex="1" GridPane.columnIndex="1"/>

    <HBox spacing="10" alignment="CENTER" GridPane.rowIndex="2"
GridPane.columnIndex="0" GridPane.columnSpan="2">
        <Button text="Войти" onAction="#handleLogin"/>
        <Button text="Отмена" onAction="#handleCancel"/>
    </HBox>

```


</GridPane>

MainView.fxml

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>

<?import javafx.scene.layout.*?>

<VBox xmlns:fx="http://javafx.com/fxml"

fx:controller="com.example.studentcontrol.ui.MainController">

<MenuBar>

<Menu text="Вид">

<MenuItem text="Тёмная тема" onAction="#setDarkTheme"/>

<MenuItem text="Светлая тема" onAction="#setLightTheme"/>

</Menu>

<Menu text="Помощь">

<MenuItem text="О программе" onAction="#showAbout"/>

</Menu>

</MenuBar>

<TabPane fx:id="tabPane" VBox.vgrow="ALWAYS">

<Tab text="Студенты" closable="false">

<fx:include source="StudentsTab.fxml" />

</Tab>

<Tab text="Предметы" closable="false">

<fx:include source="SubjectsTab.fxml" />

</Tab>

<Tab text="Оценки" closable="false">

<fx:include source="GradesTab.fxml" />

</Tab>

</TabPane>

</VBox>

StudentsTab.fxml

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>

<?import javafx.scene.control.*?>

<?import javafx.scene.layout.*?>

<VBox spacing="10" xmlns:fx="http://javafx.com/fxml"

fx:controller="com.example.studentcontrol.ui.StudentsTabController">

<padding>

<Insets top="10" right="10" bottom="10" left="10"/>

</padding>

```

<TableView fx:id="studentsTable" VBox.vgrow="ALWAYS">
    <columns>
        <TableColumn text="ID" prefWidth="50" fx:id="colId"/>
        <TableColumn text="Фамилия" prefWidth="150" fx:id="colLastName"/>
        <TableColumn text="Имя" prefWidth="150" fx:id="colFirstName"/>
        <TableColumn text="Группа" prefWidth="100" fx:id="colGroup"/>
        <TableColumn text="Дата поступления" prefWidth="120"
fx:id="colDate"/>
    </columns>
</TableView>

<GridPane hgap="10" vgap="10">
    <padding>
        <Insets top="10"/>
    </padding>

    <Label text="Фамилия:" GridPane.rowIndex="0"
GridPane.columnIndex="0"/>
    <TextField fx:id="lastNameField" GridPane.rowIndex="0"
GridPane.columnIndex="1"/>

    <Label text="Имя:" GridPane.rowIndex="0" GridPane.columnIndex="2"/>
    <TextField fx:id="firstNameField" GridPane.rowIndex="0"
GridPane.columnIndex="3"/>

    <Label text="Группа:" GridPane.rowIndex="1"
GridPane.columnIndex="0"/>
    <TextField fx:id="groupField" GridPane.rowIndex="1"
GridPane.columnIndex="1"/>

    <Label text="Дата (ГГГГ-ММ-ДД):" GridPane.rowIndex="1"
GridPane.columnIndex="2"/>
    <TextField fx:id="dateField" promptText="2024-09-01"
GridPane.rowIndex="1" GridPane.columnIndex="3"/>

    <HBox spacing="10" GridPane.rowIndex="2" GridPane.columnIndex="0"
GridPane.columnSpan="4">
        <Button text="Добавить" onAction="#handleAdd"/>
        <Button text="Обновить" onAction="#handleUpdate"/>
        <Button text="Удалить" onAction="#handleDelete"/>
    </HBox>
</GridPane>

```

```

</VBox>
SubjectsTab.fxml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox spacing="10" xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.example.studentcontrol.ui.SubjectsTabController">
    <padding>
        <Insets top="10" right="10" bottom="10" left="10"/>
    </padding>

    <TableView fx:id="subjectsTable" VBox.vgrow="ALWAYS">
        <columns>
            <TableColumn text="ID" prefWidth="50" fx:id="colId"/>
            <TableColumn text="Название" prefWidth="300" fx:id="colName"/>
            <TableColumn text="Семестр" prefWidth="100" fx:id="colSemester"/>
        </columns>
    </TableView>

    <GridPane hgap="10" vgap="10">
        <padding>
            <Insets top="10"/>
        </padding>

        <Label text="Название:" GridPane.rowIndex="0"
GridPane.columnIndex="0"/>
        <TextField fx:id="nameField" GridPane.rowIndex="0"
GridPane.columnIndex="1"/>

        <Label text="Семестр:" GridPane.rowIndex="0"
GridPane.columnIndex="2"/>
        <Spinner fx:id="semesterSpinner" editable="true" GridPane.rowIndex="0"
GridPane.columnIndex="3"/>

        <HBox spacing="10" GridPane.rowIndex="1" GridPane.columnIndex="0"
GridPane.columnSpan="4">
            <Button text="Добавить" onAction="#handleAdd"/>
            <Button text="Обновить" onAction="#handleUpdate"/>
            <Button text="Удалить" onAction="#handleDelete"/>
        </HBox>
    </GridPane>
</VBox>

```